Hooks & skills

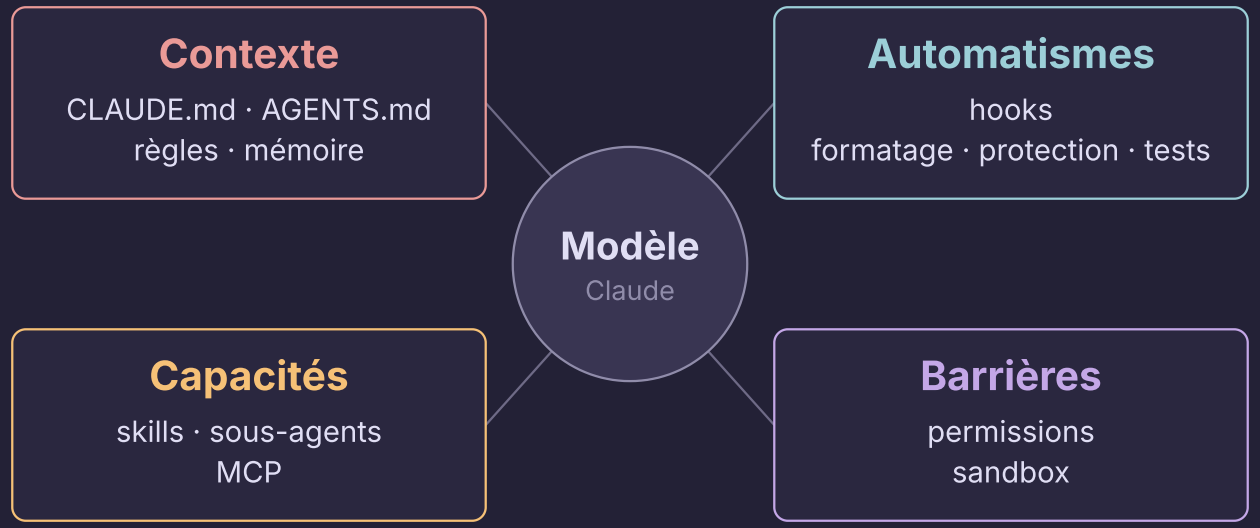
J1 après-midi

Les automatismes et les capacités du harness

Émargement de jeudi après-midi : signez en arrivant

`edusign.app/sc/SGAAPQSV`

Où on en est



Ce matin : le contexte

Premier contact avec l'agent, fenêtre de contexte, prompt précis, CLAUDE.md et une règle

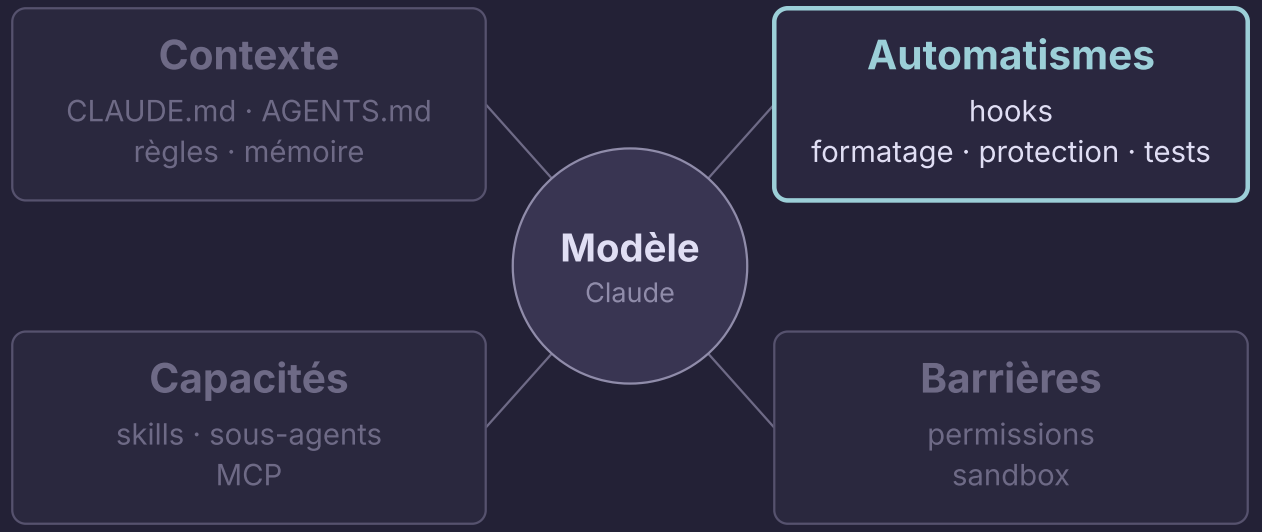
Cet après-midi : les trois autres zones

Hooks et **barrières** : ce qui doit arriver, ce qui est interdit

Skills : du savoir ou des procédures, à la demande

Hooks

Du conseil à la garantie



Conseil ou garantie

CLAUDE.md : un conseil

```
## Code style
- Formatez avec Prettier.
```

Lu par **le modèle**, parmi tout le reste de la fenêtre
→ suivi **la plupart du temps**

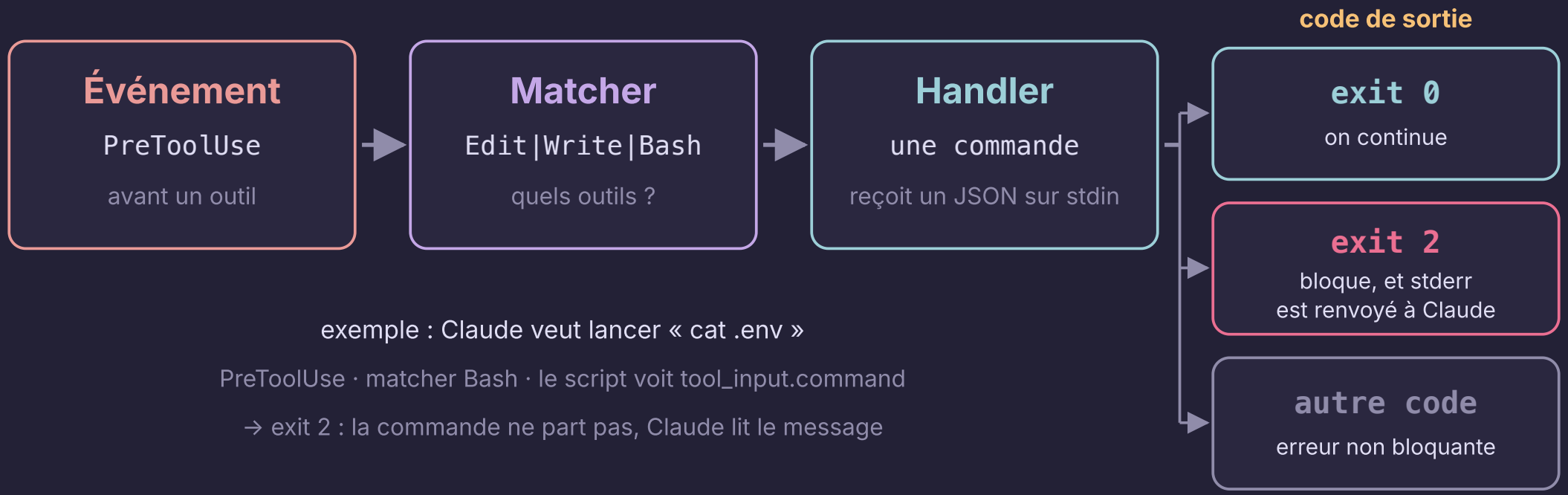
Hook : déterministe

```
npx prettier --write <fichier modifié>
```

Lancé par **Claude Code** après chaque modification
→ exécuté **à chaque fois**

Ce qui doit arriver à chaque fois ne s'écrit pas dans le CLAUDE.md, ça se branche dans un hook.

Anatomie d'un hook



exemple : Claude veut lancer « cat .env »

PreToolUse · matcher Bash · le script voit tool_input.command

→ exit 2 : la commande ne part pas, Claude lit le message

Sur stdin : `{ "tool_name": "Bash", "tool_input": { "command": "cat .env" } }`

Les événements utiles

Événement	Quand	Exemple
`SessionStart`	au démarrage, ou après une compaction (`compact`)	recharger le contexte
`UserPromptSubmit`	à chaque prompt envoyé	ajouter une information
`PreToolUse`	avant un outil : **peut bloquer**	protéger un fichier
`PostToolUse`	après un outil : ne peut plus bloquer	formater
`Stop`	quand Claude veut s'arrêter	lancer les tests
`Notification`	quand Claude vous attend	vous prévenir

33 événements au total, dont `PermissionRequest`, `PreCompact`, `SubagentStop`, `FileChanged`, `SessionEnd`.

La configuration, dans `.claude/settings.json`

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command":
          "jq -r '.tool_input.file_path' | xargs npx prettier --write"
      }]
    }]
  }
}
```

`.claude/settings.json` : versionné, pour **toute l'équipe**

`.claude/settings.local.json` : **personnel**, à ne pas committer

Un hook s'exécute avec vos droits : relisez le `.claude/` d'un repo cloné avant d'accepter la confiance.

DÉMO Les hooks de Bénéfriches

```
"hooks": {
  "PreToolUse": [{ "matcher": "Bash", "hooks": [
    { "type": "command", "command": "…/block-env-read.sh" },
    { "type": "command", "command": "…/block-hook-bypass.mjs" }
  ] }],
  "PostToolUse": [{ "matcher": "Edit|Write", "hooks": [{
    "type": "command",
    "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/format-check.sh",
    "statusMessage": "Formatting code with Prettier..."
  }] }]
}
```

Prompt : « Crée packages/shared/src/demo-format.ts avec une fonction add(a, b), écrite sur une seule ligne, sans espaces ni point-virgule. »

Source : Bénéfriches (licence MIT), .claude/settings.json, extrait remis en forme · github.com/incubateur-ademe/benefriches

Un hook Stop qui lance les tests

```
{ "hooks": { "Stop": [{
  "hooks": [{
    "type": "command",
    "command":
      "node \"${CLAUDE_PROJECT_DIR}/.claude/hooks/tests-verts.mjs\""
  }]
}] } }
```

Pas de `matcher` : `Stop` se déclenche à chaque fin de tour.

```
// .claude/hooks/tests-verts.mjs
import { execSync } from "node:child_process";
let raw = "";
for await (const c of process.stdin) raw += c;
if (JSON.parse(raw).stop_hook_active)
  process.exit(0); // déjà relancé : on lâche
try {
  execSync("npm test 2>&1");
} catch (e) {
  console.error("Tests KO :\n" + e.stdout);
  process.exit(2); // Claude doit continuer
}
```

Des tests rouges renvoient Claude au travail au lieu de le laisser s'arrêter.

Hooks Claude Code et hooks git : le pont

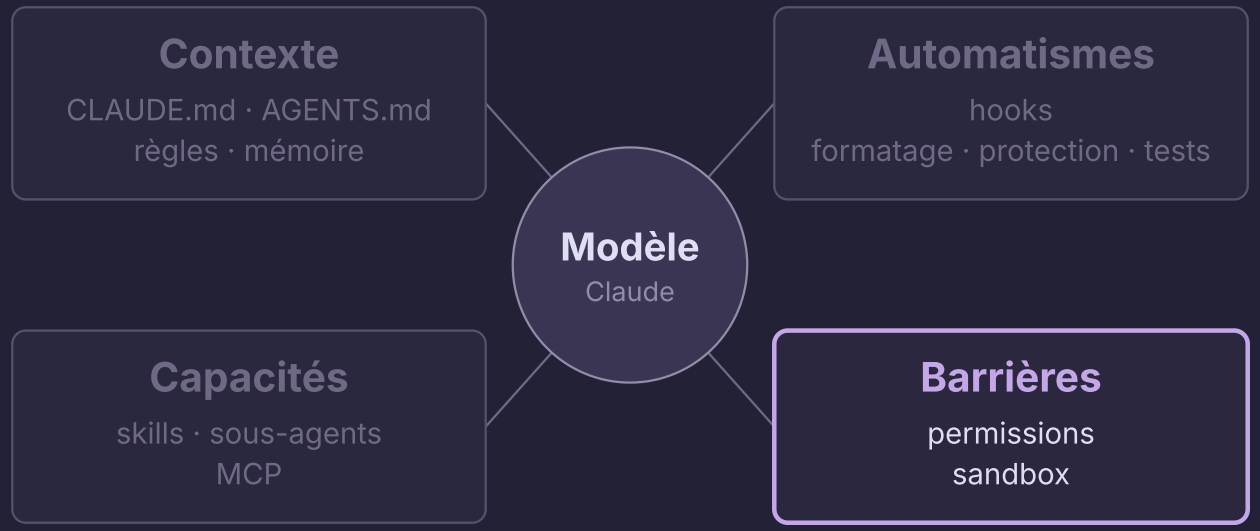
Critère	Hooks Claude Code	Hooks git
Quand	pendant la session, à chaque action de l'agent	au commit, au push
Pour qui	l'agent	tout le monde, humain ou agent
Exemples	Prettier, protection du `.env`	Husky, lint-staged, Talisman

Bénéfriches relie les deux : `block-hook-bypass.mjs` (PreToolUse sur Bash) bloque `git commit --no-verify`, `-n`, `HUSKY=0`… donc le pre-commit (Talisman, lint) tourne même si Claude tente les contournements courants.

Pas encore de hooks git ? Le skill `setup-pre-commit` (Matt Pocock, hors de son plugin) installe Husky, lint-staged, typecheck et tests au commit.

Barrières

Ce qu'on interdit, et ce qu'on isole



EXERCICE Protégez votre `.env`

Étape	Testez
1. Règle `deny` (5 min) : le bloc `permissions` ci-dessous	« Lis le fichier .env » → refusé
2. Hook (15 min) : `protect-env.mjs` du livret dans `.claude/hooks/`, puis le bloc `hooks`	« Affiche le .env avec cat » → bloqué
3. La faille (5 min) : autre commande, autre chemin…	notez ce qui passe

```
"permissions": { "deny": ["Read(**/.env)", "Edit(**/.env)"] },
"hooks": { "PreToolUse": [{
  "matcher": "Read|Edit|Write|Bash|PowerShell|Grep",
  "hooks": [{ "type": "command", "command":
    "node \"${CLAUDE_PROJECT_DIR}/.claude/hooks/protect-env.mjs\"" }]
}] }
```

.NET : `protect-env.mjs` couvre aussi `appsettings.Development.json` et `secrets.json`.

La solution de Bénéfriches

`settings.json` : 7 règles

```
"deny": [
  "Read(.env)",
  "Read(**/.env)",
  "Read(**/.env.local)",
  "Read(**/.env.production)",
  "Edit(**/.env)",
  "Edit(**/.env.local)",
  "Edit(**/.env.production)"
]
```

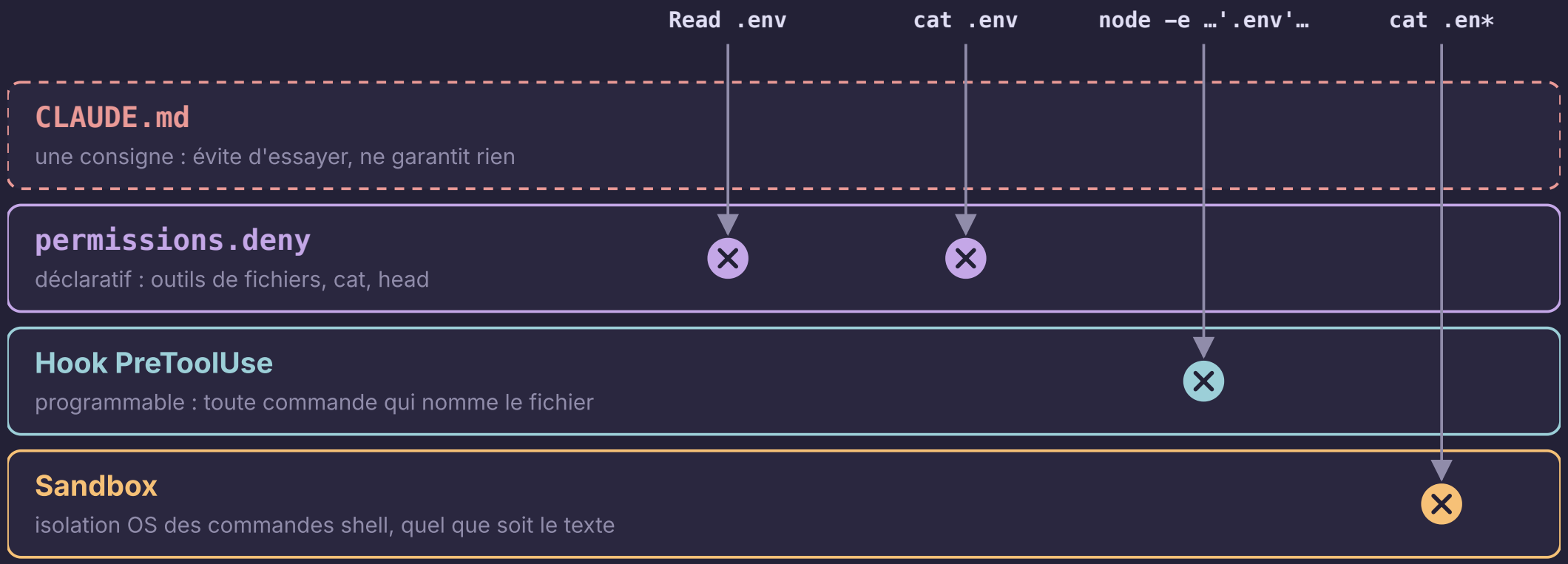
`block-env-read.sh` : PreToolUse sur Bash

```
cmd=$(jq -r '.tool_input.command // ""')
sanitized=$(printf '%s' "$cmd" | sed -E \
  's/\.env\.(example|e2e|test)/__ENVOK__/g')
if printf '%s' "$sanitized" \
   | grep -qE '\.env($|[^a-zA-Z0-9])'; then
  echo "Blocked: this command ..." >&2
  exit 2
fi
exit 0
```

Les règles `deny` visent les fichiers ; le hook lit **le texte de chaque commande Bash**.

Source : Bénéfriches (licence MIT), .claude/settings.json et .claude/hooks/block-env-read.sh, extraits remis en forme

Défense en profondeur

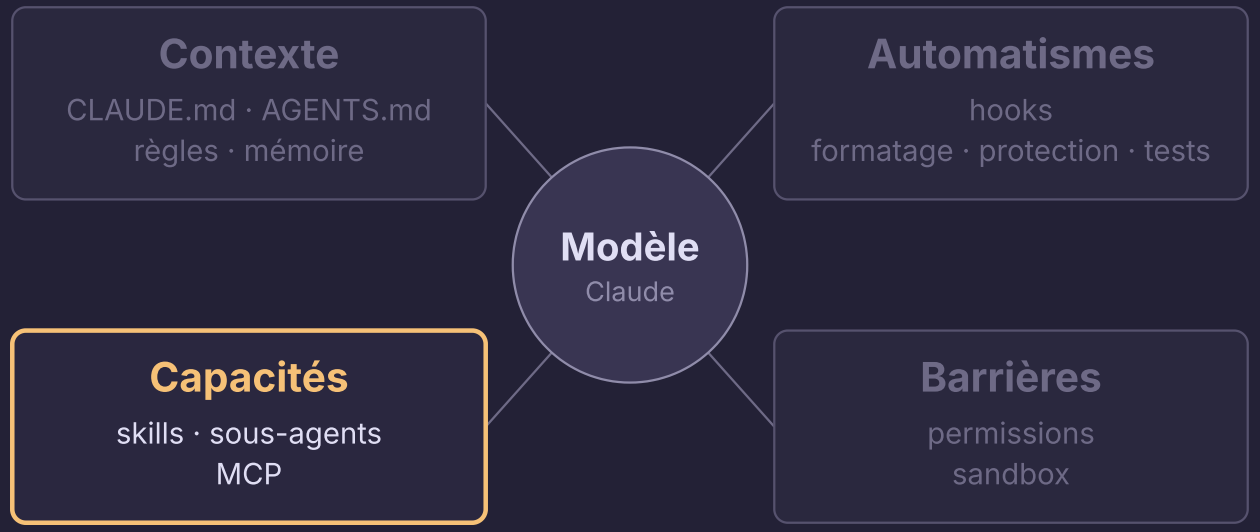


chaque tentative s'arrête à la première couche qui la voit

Chaque couche couvre les trous de la précédente.

Skills personnalisés

Du savoir ou des procédures, à la demande



Quand créer un skill

> Créez un skill quand vous recollez sans cesse le même playbook ou la même procédure dans le chat, ou quand une section de votre CLAUDE.md est devenue une procédure plutôt qu'un fait.

D'après la documentation d'Anthropic sur les skills (paraphrase)

Une procédure

« Créer une migration de base de données en respectant nos conventions »

Une checklist

« Faire une revue de code selon notre checklist »

Un savoir

« Comment marche notre ancien module de facturation », utile seulement quand on y touche

Structure et emplacements

```
mon-skill/
├── SKILL.md          # obligatoire
├── reference.md      # détails, lus à la demande
├── examples/         # exemples de sortie
└── scripts/          # scripts que le skill lance
```

Portée	Emplacement
Personnel	`~/.claude/skills/<nom>/`
Projet (versionné)	`.claude/skills/<nom>/`
Plugin	`skills/` du plugin

Bénéfriches : 13 skills dans `.agents/skills/`, reliés dans `.claude/skills/` par des liens symboliques. Claude Code et Codex lisent les mêmes.

Anatomie d'un SKILL.md

```
---
name: create-database-migration
description: Create Knex database migrations.
  Use when adding, modifying or removing columns.
effort: low
allowed-tools: Bash, Read, Write
---
## Étapes
1. pnpm --filter api knex:new-migration <nom>
2. Écrire up() et down(), puis tableTypes.d.ts
## Pièges
- Pas de knex.transaction() : déjà transactionnel
```

`description` : ce que fait le skill et quand l'utiliser. C'est elle qui déclenche.

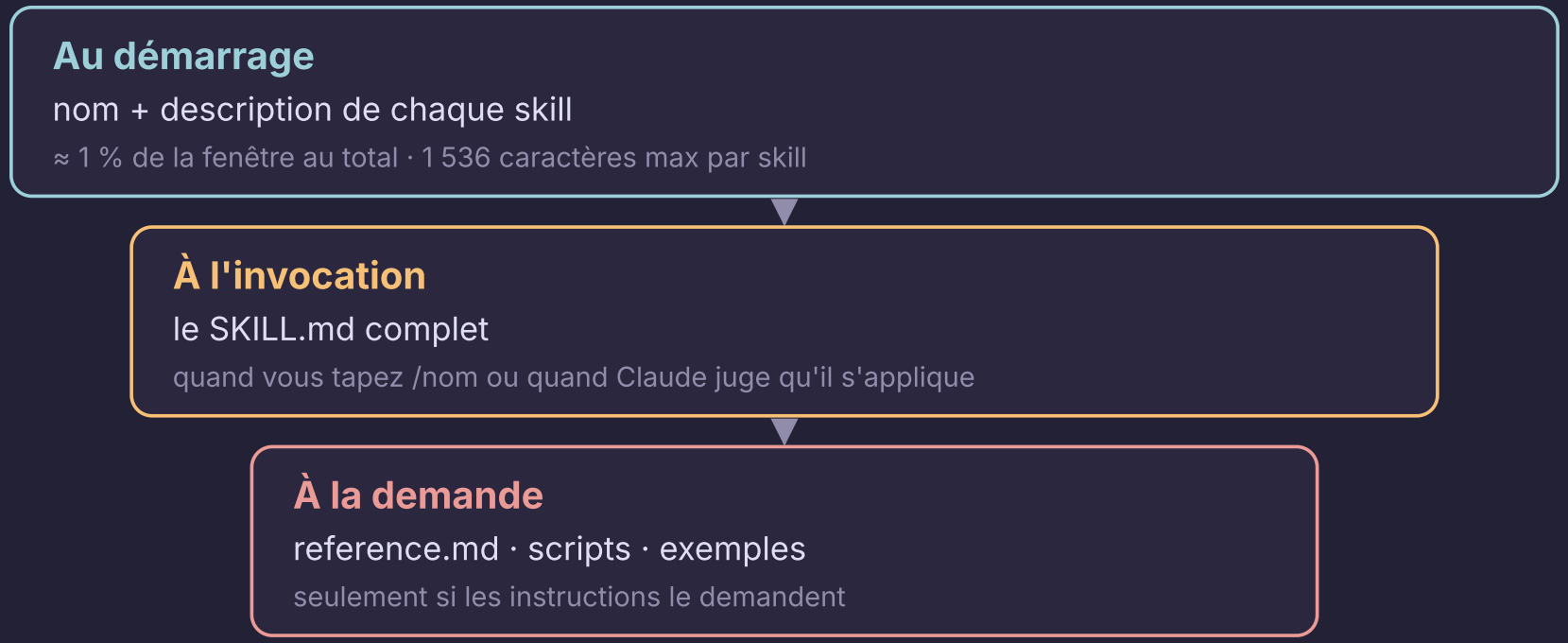
`disable-model-invocation: true` : seul l'humain peut l'invoquer.

`allowed-tools` : pré-approuve ces outils pendant le tour qui invoque le skill. Ne restreint rien.

`context: fork` : le skill s'exécute dans un sous-agent.

Inspiré de .agents/skills/create-database-migration/SKILL.md de Bénéfriches (licence MIT), abrégé et traduit en partie

Le chargement progressif



`code-reviewer` (Bénéfriches) : `reference/api.md` si `apps/api/**` a changé, `reference/web.md` si c'est `apps/web/**`, les deux si c'est le package partagé.

Qui déclenche le skill

Configuration	Vous (`/nom`)	Claude (seul)	Exemple
par défaut	oui	oui	revue de code
`disable-model-invocation: true`	oui	non	déployer, committer : des effets de bord
`user-invocable: false`	non	oui	des conventions que Claude applique en arrière-plan

Claude ne déclenche jamais votre skill ? C'est presque toujours la description.

DÉMO Un skill de Bénéfriches : `create-database-migration`

Prompt, sans nommer le skill :

> J'ai besoin d'ajouter une colonne comment, texte optionnel, à la table des sites.

Ce qu'on observe :

1. Claude choisit le skill **tout seul**, grâce à sa description
2. il suit la procédure : `pnpm --filter api knex:new-migration …`, `up()` et `down()`, `tableTypes.d.ts`
3. il évite le piège des transactions écrit dans le SKILL.md

Skill ou CLAUDE.md ?

Critère	CLAUDE.md	Skill
Chargement	toujours : coût permanent	à la demande
Contenu	ce qui sert à chaque session : faits, conventions, commandes	ce qui sert à une tâche : un savoir ou une procédure
Portée	tout le projet	une tâche

Toujours pertinent : CLAUDE.md. Parfois pertinent : skill.

Pause

Reprise dans 15 minutes

Écrire un bon skill

1. Une seule responsabilité
`create-database-migration` ne fait que des migrations.

2. Pas d'évidence
Seulement ce qui sort Claude de ses habitudes sur **ce** projet.

3. Une section Pièges (Gotchas)
Enrichie à chaque usage raté.

4. Un SKILL.md court
Moins de 500 lignes ; le détail dans des annexes.

5. Commencer simple
`/grill-me` de Matt Pocock tient en une ligne, qui appelle un skill d'une trentaine de lignes : des sessions de dizaines de questions.

Partager et faire vivre en équipe

1. Dans le repo : versionnez `.claude/skills/`, toute l'équipe l'a au prochain pull.

2. Pour plusieurs repos : un plugin dans une marketplace, activé dans `.claude/settings.json`

```
"extraKnownMarketplaces": {
  "outils-equipe": { "source": {
    "source": "github", "repo": "votre-org/claude-plugins" } }
},
"enabledPlugins": {
  "conventions-angular@outils-equipe": true
}
```

3. Au-delà de Claude : un standard ouvert. Bénéfriches partage ses skills avec Codex.

Un skill peut exécuter du code : relisez-le avant de l'installer.

Composer des skills en workflow

1. `/grill-me` clarifier l'intention → 2. `/to-spec` écrire la spec → 3. `/to-tickets` découper, avec les dépendances → 4. `/implement` implémenter, test d'abord

La chaîne de Matt Pocock : chaque skill produit ce que le suivant consomme.

Chaque skill fait une chose ; la puissance vient de la composition.

Qui fait quoi

Brique	Ce que c'est	Qui la déclenche	Exemple
Commande intégrée	une fonction de Claude Code	vous	`/context`, `/compact`
Skill	un savoir ou une procédure, chargé à la demande	vous ou Claude	`create-database-migration`
Sous-agent	une instance à contexte isolé, qui rend un résultat	Claude ou vous	`security-reviewer` (Bénéfriches)
MCP	un connecteur vers un outil externe	Claude	context7, un navigateur, GitHub

Et toujours : **CLAUDE.md** décrit, **les hooks** garantissent.

Des plugins pour vos stacks

Plugin	Ce qu'il apporte
`typescript-lsp`, `csharp-lsp`, `jdtls-lsp` (Java), `pyright-lsp` (Python)	l'intelligence de code de votre langage : diagnostics, navigation
`hookify`	créer un hook en une phrase : « préviens-moi quand j'utilise rm -rf »
`skill-creator`	créer, améliorer et évaluer un skill : **on l'installe maintenant**

```
/plugin install <nom>@claude-plugins-official
```

EXERCICE Créez votre skill

Phase	Durée	À faire
0. Installer	5 min	`/plugin install skill-creator@claude-plugins-official`
1. Choisir	10 min	une tâche répétitive (idées : slide suivante)
2. Créer	20 min	à la main avec le modèle du livret, ou avec `skill-creator`
3. Déclencher	15 min	sur un cas réel : Claude l'invoque-t-il seul ? Sinon, retravaillez la description
4. Test croisé	10 min	votre binôme demande la tâche sans nommer le skill
5. Commit	5 min	`.claude/skills/<nom>/SKILL.md`

Livrable : un skill versionné dans votre repo.

EXERCICE Idées et modèle

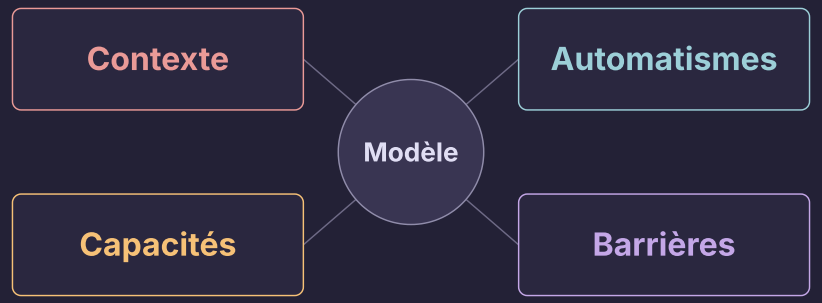
- **Angular** : composant + test, service HTTP + mock, revue d'accessibilité
- **.NET** : migration Entity Framework, endpoint + test xUnit, mapping DTO
- **Java Spring** : controller + test MockMvc
- **Python** : script + test pytest
- **Transverse** : description de PR, changelog, revue selon vos conventions

Et un savoir, sans étapes : le fonctionnement d'un module.

```
---
name: mon-skill
description: Ce que fait le skill.
  À utiliser quand <mots de l'équipe>.
---
## Objectif
Ce que le skill produit.
## Étapes
1. …
2. Vérifier : <commande de test>
## Pièges
- Ce que Claude rate sur CE projet
```

Ce qu'il faut retenir du J1

Brique	Chargée	Ce qu'on y met
CLAUDE.md	à chaque session	ce qui sert toujours : commandes, conventions, pièges
Règles `.claude/rules/`	selon le dossier	ce qui ne vaut que pour une partie du code
Skills	à la demande	un savoir ou une procédure, utile à une tâche
Hooks	à coup sûr, hors du contexte	ce qui doit arriver à chaque fois
Permissions	à chaque appel d'outil, hors du modèle	ce que l'agent ne doit jamais faire



Les quatre zones du harness en place ; demain : composer et paralléliser.

Ce soir : votre ticket fil rouge

Une tâche réelle de votre backlog, qui soit :

Critère	Pourquoi
Une demi-journée à deux jours de travail	assez gros pour être découpé, assez petit pour avancer demain
Découpable en 2 ou 3 morceaux indépendants	pour les faire en parallèle demain après-midi
Vérifiable : pas de tests ? l'agent les écrira d'abord	sans critère, l'agent ne sait pas quand il a fini
Sur votre repo, montrable en salle	on le décompose ensemble

On le valide en 5 minutes demain à 9h40.

Plan d'adoption 30 jours : ligne 2

Le hook que je mets en place dans mon équipe

Le skill que je partage

J1 matin ligne 1 : CLAUDE.md et une règle
J1 après-midi **ligne 2 : un hook, un skill**

Exemple : « le hook `.env` dans tous nos repos front » ; « le skill de revue selon nos conventions Angular ».

Tour de table

1

Un truc que vous **retenez**

2

Un truc qui vous a **manqué** ou frustré

3

Une chose que vous allez **essayer** demain matin

Une minute et demie chacun.

Demain

Composer des workflows & orchestrer en parallèle

J2 · reprise à 9h30 · validation des tickets fil rouge à 9h40